C++ Enumerations

Mastering Symbolic Constants & Fixed Sets

Topic: User-Defined Data Types (enum)

Section: Logical Control & Data Clarity

1. Understanding Enums

In programming, clarity is just as important as logic. Often, we use numbers to represent states. For example, we might say `0 = Stopped`, `1 = Starting`, and `2 = Running`. However, reading `if(status == 2)` later in a 1,000-line file is confusing.

An **Enumeration (enum)** is a user-defined data type that represents a fixed set of named constant values. It allows us to give meaningful names to these "magic numbers."

> **Key Benefit:** Enums make your code "self-documenting." Instead of checking a manual to see what "2" means, you can simply write `RUNNING`.

The Real-World Logic

Enums are ideal for sets that are:

- **Finite:** There is a clear beginning and end.
- **Fixed:** The options rarely change (e.g., Months of the year, Player roles).
- **Mutually Exclusive:** A variable usually only holds one value from the set at a time.

2. Creating an Enum

To define an enumeration, use the `enum` keyword followed by the name of the type and its possible values inside curly braces.

```
enum Level {
  LOW,
  MEDIUM,
  HIGH
};
```

Anatomy of the Definition

- **enum:** The keyword used to declare the type.
- **Level:** The name of your new data type.
- **LOW, MEDIUM, HIGH:** These are called Enumerators. They are constants.

Creating Variables

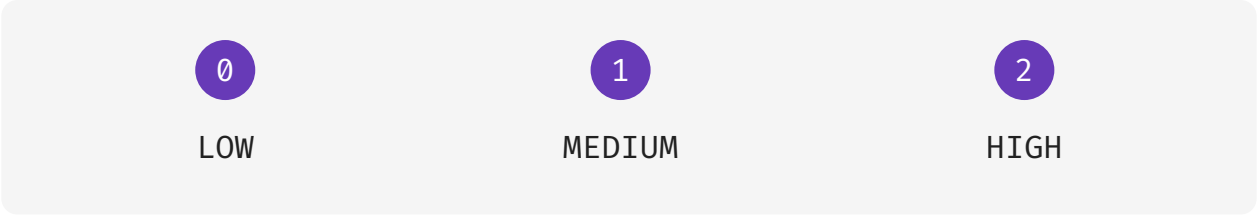
Once the type is defined, you can create a variable to hold one of those values:

```
Level myAchievement = HIGH;

if (myAchievement == HIGH) {
    cout << "Congratulations!";
}
```

3. The Integer Relationship

Internally, C++ treats enums as integers. By default, the computer starts counting from zero for the first item and increases by one for each subsequent item.




```
enum Status { ACTIVE, INACTIVE, BLOCKED };

cout << ACTIVE;   // Outputs: 0
cout << INACTIVE; // Outputs: 1
cout << BLOCKED;  // Outputs: 2
```

Note: Because they are internally integers, enums are extremely fast for the CPU to compare, making them more efficient than using `string` comparisons.

4. Customizing Enum Values

You are not forced to start at `0`. You can assign specific integer values to any or all of the enumerators.

Specific Assignment

```
enum HTTP_Code {
  OK = 200,
  NOT_FOUND = 404,
  SERVER_ERROR = 500
};
```

Automatic Increment Logic

If you assign a value to one item but not the others, C++ will simply continue counting from that number for the following items.

```
enum Score {
  MIN = 10,
  MID,  // Automatically becomes 11
  MAX   // Automatically becomes 12
};
```

This is useful when you want to establish a sequence but don't want to type every single number manually.

5. Enums and Switch Statements

Enums and `switch` statements are a perfect match. Together, they provide one of the cleanest ways to handle different program states or user roles.

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN };

Day today = FRI;

switch (today) {
    case SAT:
    case SUN:
        cout << "It's the weekend!";
        break;
    default:
        cout << "Work day...";
        break;
}
```

Why this is better than "if-else":

- **Readability:** The intent of each case is obvious immediately.
- **Maintenance:** If you add a value to the Day enum, you can easily add a new case to your switch.

6. Enums vs. Multiple Constants

You might wonder: "Why not just use `const int`?" While both store constant values, enums have distinct advantages.

The Const Way (Inefficient)

```
const int RED = 0;
const int YELLOW = 1;
const int GREEN = 2;
```

The Enum Way (Professional)

```
enum TrafficLight { RED, YELLOW, GREEN };
```

Primary Advantages:

1. **Grouping:** Enums group related constants together under a single type name.
2. **Type Safety:** If a function expects a `TrafficLight`, you cannot accidentally pass it a HTTP_Code, even if both are technically integers.
3. **Debugging:** Some debuggers can show the name (RED) instead of the number (0).

7. Best Practices & Pitfalls

Naming Conventions

Most C++ developers follow these naming rules for enums:

- **Type Name:** Use PascalCase (e.g., `GameDifficulty`).
- **Values:** Use ALL_CAPS to signal that these are constants that do not change (e.g., `EASY`, `HARD`).

Common Mistakes

No String Storage: You cannot store a string directly in an enum.
`enum Color { RED = "Red" }; // ❌ Error!`
Enums are for numeric mapping only.

Scope Collision: In standard enums, you cannot have two different enums with the same member names in the same scope.

8. Code Challenge Lab

Practice what you've learned to solidify your understanding of Enumerations.

Challenge: The User Access System

1. Create an enum named `UserRole` with values: `GUEST`, `MEMBER`, and `ADMIN`.
2. Assign the values 10, 20, and 30 to them respectively.
3. Create a variable for a user and set it to `MEMBER`.
4. Write a switch statement that prints "Welcome, Admin!" only if the role is 30.

9. Summary

- **Enums** represent a fixed set of named integer constants.
- Values start at **0** by default but can be customized.
- Using enums eliminates **"Magic Numbers"** from your code.
- They work seamlessly with **switch statements** for state management.
- Enum values should be **UPPERCASE** for better visibility.